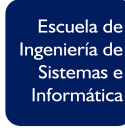


 	Universidad Industrial de Santander Facultad de Ingenierías Fisicomecánicas Escuela de Ingeniería de Sistemas e Informática	 
Asignatura: Estructuras de Datos y Análisis de Algoritmos	Código: 22955	Grupo: B1/C1
Docente: Laura Viviana Galvis Carreño	Correo: lavigal@uis.edu.co	

Taller 1. Introducción a las Estructuras de Datos y Recursividad

Agosto 28 de 2025

Instrucciones: El informe del taller debe subirse al moodle, antes del **4 de Septiembre a las 8:00 pm**. Informe enviado después de la fecha y hora estipulada no será tenido en cuenta.

El informe debe incluir el código en Java programado en cada uno de los ejercicios cuando se indique y como se indique, así como los resultados de la ejecución en forma de captura de pantalla. Se debe crear **un archivo comprimido** con el nombre **"EDA_B1/C1_Taller1_Nombre_Apellido"**, en caso de que se pida un proyecto y un **archivo PDF del informe (resultado de la plantilla de Latex/Overleaf)** con el mismo nombre **"EDA_B1/C1_Taller1_Nombre_Apellido"**.

1. SOBRE INTRODUCCIÓN A LAS ESTRUCTURAS DE DATOS

1.1 Manejo y comprensión de los principios de POO: Se quiere modelar la clase Animales vertebrados, con los descendientes Mamífero, Ave y Pez. A su vez, de la clase Mamífero descenden las clases Gato y Perro; de la clase Ave descenden las clases Gallo y Pato; y de la clase Pez descenden las clases Tiburón y Bocachico.

Todos los animales tendrán como mínimo, los atributos comunes: *color* y *numeroDePatatas*, y los métodos comunes *hacerMovimiento()*, *hacerSonido()*, que describirán su forma de moverse y onomatopeya, y *tienePatatas()* y *tieneDientes()*, que verificarán si el Animal tiene patas y/o dientes.

Entregables:

- Diagrama de clases UML, representando la herencia entre clases, el *modificador de acceso* de cada atributo y método, así como el *tipo de datos*.
- Código programado en Java de cada una de las clases diseñadas incluyendo *getters/setters*. Incluir comentarios.

- Clase *Main.java*, con método `public static void main()` que crea una instancia de cada una de las clases, e imprime en consola su forma de moverse, su onomatopeya, su color, su número de patas y si tiene o no tiene dientes. Adicionalmente, si el objeto pertenece a la clase Ave, debe imprimirse si puede volar o no; y si pertenece a la clase Pez, debe imprimirse si vive en el mar o no.
- Captura de pantalla de la ejecución del código.

1.2. Crear una función que ordene un vector de tamaño fijo de manera ascendente o descendente usando la clase Arrays. La función debe recibir dos parámetros: el primer parámetro es el vector de tamaño fijo (de tipo numérico o String) y el segundo parámetro es un String que describe la forma en cómo se debe ordenar el vector de entrada ("ascendente" o "descendente"). Incluir código (*lstlisting*) y captura de ejecución en el informe.

```

1  public static int[] ordenarNumeros (int[] vector, String tipoOrden){
2      // si tipoOrden es "ascendente"
3
4      // si tipoOrden es "descendente"
5      return vector;
6  }
```

```

1  public static String[] ordenarStrings (String[] vector, String tipoOrden){
2      // si tipoOrden es "ascendente"
3
4      // si tipoOrden es "descendente"
5      return vector;
6  }
```

1.3. Se busca diseñar un programa que reciba notas de estudiantes (enteros entre 1 y 100 puntos) para mostrar estadísticas de rendimiento. Se planea que el programa recibirá las notas de 200 estudiantes. Si el programa solo requiere imprimir las frecuencias por encima de 50 puntos. ¿Cuál sería la estructura de datos eficiente y necesaria para que el programa logre su objetivo? Justificar su respuesta y motivos de su selección.

- Un Array con un tamaño de 200 int
- Un Array con un tamaño de 100 int
- Un Array con un tamaño de 50 int
- Un Array dinámico de 250 int

1.4. ¿Cuáles de las siguientes declaraciones son verdaderas sobre la herencia en Java?

1. Los métodos `protected` son `final`
 2. Los métodos `private` son `final`
 3. No es posible `@Override` métodos `private`
 4. Un miembro `protected` es accesible dentro del `package` donde se define y por las clases heredadas fuera del `package`
- a. Solo 2 y 4
 - b. 1, 2 y 4
 - c. 1, 3 y 4
 - d. 2, 3 y 4

2. SOBRE RECURSIVIDAD

2.1. Reemplace `//Write your code here` en el siguiente código; es decir, implemente el método recursivo `encontrarSuma()`. Este método calcula la suma de los enteros positivos menores e iguales a `n`. Por ejemplo, `encontrarSuma(4)` debe imprimir 10. Reporte el código (*lstlisting*) y captura de pantalla de su ejecución.

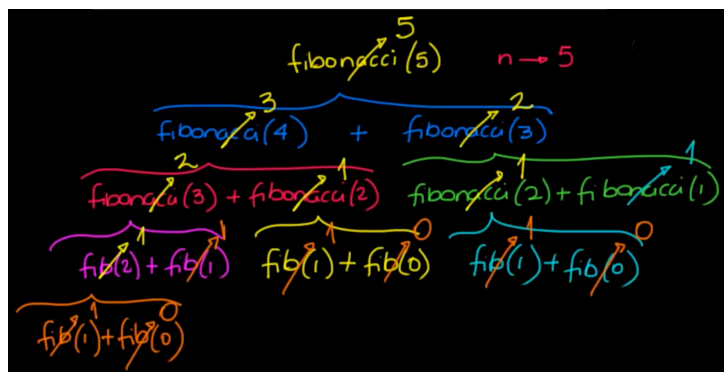
```
public class Recursion {  
  
    public static void main(String[] args) {  
        System.out.println(encontrarSuma(8));  
    }  
  
    public static int encontrarSuma(int n){  
  
        // Write your code here  
    }  
}
```

2.2. ¿Qué hace el siguiente método recursivo?. ¿En sus palabras, cuál es la recursión que resuelve? Explique.

- a. $a + (b - 1)$
- b. $a * b$
- c. $a + a * b$
- d. a^b

```
static int hacer(int a, int b){  
    if (b == 0){  
        return 0;  
    }  
    else{  
        return (a + hacer(a,b-1));  
    }  
}
```

2.3. **Fibonacci optimizado:** Uno de los ejemplos que vimos en la clase sobre recursión, fue el método de calcular la secuencia de Fibonacci: 0, 1, 1, 2, 3, 5, ... Discutimos que el método recursivo de Fibonacci tiene un problema cuando se trata de **eficiencia**, ya que si llamamos a **fib(5)**, la función recursiva llama una vez a **fib(4)**, dos veces a **fib(3)**, tres veces a **fib(2)**, cinco veces a **fib(1)** y tres veces a **fib(0)**, como se muestra en la figura.



Pregunta: Diseñe y programe en Java una mejor estrategia recursiva para calcular el número de fibonacci. En particular, cree una función **fibonacciOptimizado()**, de tal manera que cuando se llame a **fibonacciOptimizado(5)**, solo llame **una única vez** a la función **fibonacciOptimizado(4)**, **fibonacciOptimizado(3)**, **fibonacciOptimizado(2)**, **fibonacciOptimizado(1)** y **fibonacciOptimizado(0)**.

Incluir código (*lstlisting*) y captura de ejecución en el informe.